

**MAMUZA ENGINEERING**

TRAINING & OUTREACH DEPARTMENT

FULLY SPONSORED OUTREACH PROGRAM

# Combined Session Notes

## Sessions 1 & 2 — C Programming Foundations & Your First Embedded Program

From variables and loops in plain C, to breadboarding and blinking your ESP32's onboard LED with Embedded C.

WEEK 1 · SESSIONS 1 – 2

info@mamuzaengineering.com | 0745 927 033

## Recap of Our Previous Session

---

Before we dive into today's material, let's quickly reconnect with what we covered together in our first meeting — Introduction to IoT & Embedded Systems.

### What We Already Covered

- IoT is a network of everyday physical objects fitted with sensors and connectivity so they can sense, communicate, and sometimes act — with little or no human involvement.
- Every IoT system is built from 4 layers: Perception (sensing/acting) → Network (connectivity) → Middleware (processing/storage) → Application (user interface).
- An embedded system is a small, purpose-built computer inside a larger device, doing one job reliably — made of 3 building blocks: Hardware (MCU/MPU + memory + I/O), Physical Interfaces (sensors = inputs, actuators = outputs), and Firmware (the logic).
- A Microcontroller (MCU) is an all-in-one, low-power chip that sits at the edge, close to sensors — while a Microprocessor (MPU) is a more powerful standalone CPU that acts as a gateway and handles heavy compute.
- We met our board for this program: the ESP32 — a 32-bit dual-core MCU with built-in Wi-Fi and Bluetooth, chosen because it's affordable, low power, and network-ready out of the box.

### KEY IDEA

Keep this picture in your head as we go: today we are working entirely inside the Perception Layer and inside the Hardware/Firmware building blocks of the embedded system — we are about to start writing the actual firmware that will run on that ESP32 brain.

## What We'll Cover Today

---

Today's class combines two sessions into one. We are moving from theory into practice — by the end of today, you will have written real code and made real hardware respond to it.

### Part 1 (Session 1) — Foundations of C Programming

Before we can program the ESP32, we need a shared language. In this part, we build the vocabulary and grammar of C from the ground up — variables, data types, comments, statements, functions, conditions, and loops — always with an eye on how each idea will be reused the moment we move into Embedded C.

### Part 2 (Session 2) — From C to Embedded C: Blinking the ESP32

With the language fundamentals in hand, we apply them immediately. You will learn how a breadboard works, meet the ESP32's pinout, understand the anatomy of an Arduino/ESP32 sketch, and then write, upload, and run your very first Embedded C program — making the onboard LED blink on and off.

#### REAL-LIFE APPLICATION

This mirrors exactly how professional embedded engineers are trained: language fundamentals first, then immediately applied to real hardware — because code that cannot control a physical device is only half the skill.

# Foundations of C Programming

**OBJECTIVES — BY THE END OF THIS PART, YOU WILL BE ABLE TO**

- Explain why C is the language of choice for embedded systems
- Write and read comments in a C program
- Declare variables using the correct data type for the value being stored
- Write statements using arithmetic, comparison, and assignment operators
- Write and call your own functions
- Control the flow of a program using if / else conditions
- Repeat actions using for and while loops

## 1.1 Why We Start With C

C is a fast, low-level programming language that gives the programmer close control over how the computer's hardware and memory are used. That combination of speed and control is exactly why C (and its extension, Embedded C) is the language of choice for microcontrollers like the ESP32.

**REAL-LIFE APPLICATION**

Think of C the way you'd think of learning grammar before writing a novel. You need to be fluent in variables, loops, and functions before you can write the "story" of a working IoT device — the ESP32 firmware you'll build across this program.

## 1.2 Comments

A comment is a note written for humans, and it is completely ignored by the compiler. Comments explain why code exists, not just what it does, and they are essential once your programs grow beyond a few lines.

```
// This is a single-line comment
/* This is a
   multi-line comment */
```

**REAL-LIFE APPLICATION**

A comment is like a sticky note left on a recipe card — "don't overmix, or the cake will be dense." The oven never reads that note, but the next cook (or future-you) definitely will.

## 1.3 Variables & Data Types

A variable is a named container that holds a value. In C, every variable must be declared with a specific data type, which tells the compiler what kind of value — and how much memory — to expect.

| Type  | Stores             | Example                   |
|-------|--------------------|---------------------------|
| int   | Whole numbers      | int score = 5;            |
| float | Decimal numbers    | float temperature = 24.5; |
| char  | A single character | char grade = 'A';         |
| bool  | True or false      | bool isOn = true;         |

#### REAL-LIFE APPLICATION

A variable is like a labelled storage box in your room. "int score = 5;" is a box labelled score containing the number 5. The data type is the size and kind of box — a box built for whole numbers is a different shape from one built for decimals or a single letter.

#### KEY IDEA

In Embedded C, choosing the right data type matters even more than on a laptop — microcontrollers often have only a few kilobytes of memory, so picking the smallest box that still fits the job saves precious space.

## 1.4 Statements & Operators

A statement is one complete instruction, and in C every statement ends with a semicolon ( ; ). Operators are the symbols that act on your data:

- Arithmetic operators — +, -, \*, /, % (modulus / remainder)
- Comparison operators — ==, !=, <, >, <=, >=
- Logical operators — && (AND), || (OR), ! (NOT)
- Assignment operator — = (assign a value to a variable)

```
int temperature = 24;
temperature = temperature + 1;    // temperature is now 25
bool isHot = temperature > 30;    // comparison operator
```

#### REAL-LIFE APPLICATION

Operators are the verbs of your code's sentences. "temperature = temperature + 1;" reads just like "add one more degree to what we already have" — the same logic a smart thermostat firmware runs every time it samples the room.

## 1.5 Functions

A function is a reusable, named block of code that performs a specific job. You can send it information (parameters), and it can hand information back (a return value).

```
int addNumbers(int a, int b) {
    return a + b;
}
```

#### REAL-LIFE APPLICATION

A function is like a blender. You feed it inputs — the ingredients — it does one specific job, and hands you back an output — the smoothie. You don't rebuild the blender every time you want a smoothie; you just call it again.

#### KEY IDEA

In Embedded C, two functions are reserved and required in every single sketch: setup() and loop(). We'll meet them properly in Part 2 — but notice they're just ordinary functions, exactly like addNumbers() above.

## 1.6 Conditional Statements — if / else

Conditions let your program make decisions: run one block of code if something is true, and a different block if it isn't.

```
if (temperature > 30) {
    Serial.println("It's hot!");
}
```

```
} else {  
  Serial.println("Temperature is normal.");  
}
```

#### REAL-LIFE APPLICATION

This is precisely how a thermostat decides what to do: "if the room is too cold, turn on the heater — else, turn it off." That single if/else block is the entire brain of a basic smart thermostat.

## 1.7 Loops — for and while

Loops repeat a block of code, either a fixed number of times (for) or for as long as a condition stays true (while).

```
for (int i = 0; i < 5; i++) {  
  Serial.println(i); // prints 0,1,2,3,4  
}
```

#### REAL-LIFE APPLICATION

A loop is like a security guard doing hourly rounds — the same check, repeated for as long as the shift lasts. Every IoT device's loop() works the same way: check the sensor, decide, act, and do it again — forever, for as long as the device has power.

#### KEY IDEA

You don't need to master all of this in one sitting. Recognise these building blocks — variables, statements, functions, conditions, loops — because every single ESP32 program you write in this program is built from exactly these pieces.

## From C to Embedded C: Blinking the ESP32

### OBJECTIVES — BY THE END OF THIS PART, YOU WILL BE ABLE TO

- Explain what a breadboard is and how its rows and columns are electrically connected
- Identify the ESP32's pinout and key hardware safeguards
- Explain the anatomy of an Arduino/ESP32 sketch: `setup()` and `loop()`
- Use `pinMode()`, `digitalWrite()` and `delay()` to control real hardware
- Write, upload, and run your first Embedded C program on the ESP32

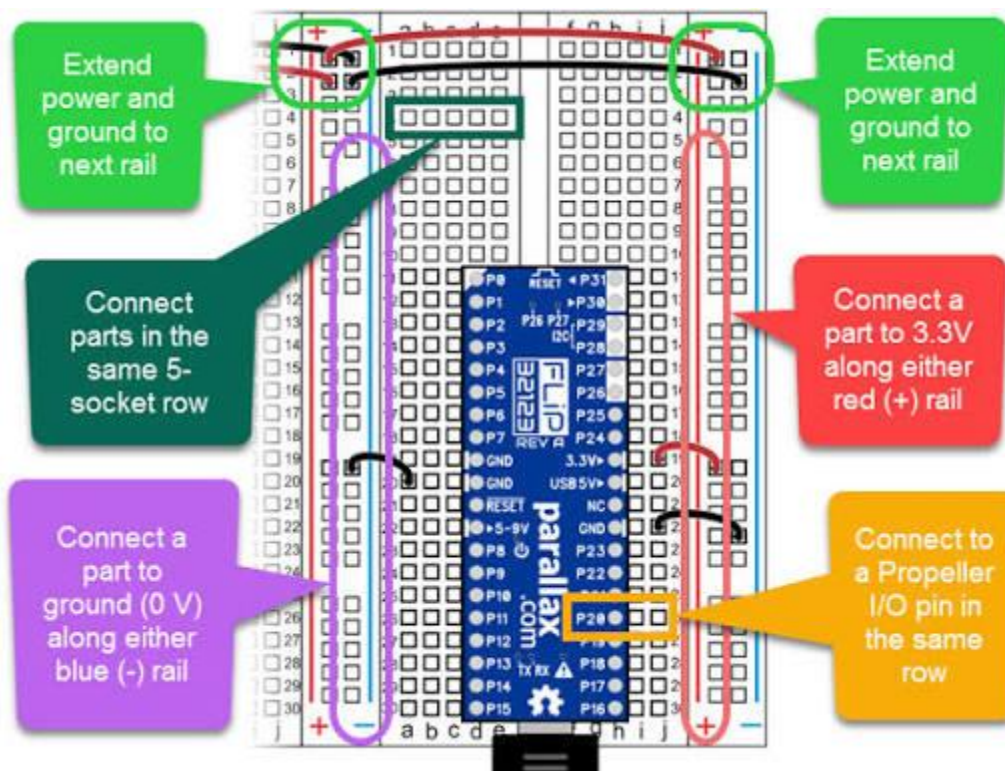
### 2.1 Breadboard Basics

A breadboard is a reusable prototyping board that lets you connect components together without soldering. Two features matter most:

- Power rails — the two outer strips (usually marked + and –, often red and blue) run the full length of the board and carry power and ground to wherever you need them.
- Terminal strips — the inner section is grouped into short vertical strips of 5 holes. Every hole in one strip is electrically connected to the others in that same strip, but strips are not connected across the centre gap unless you bridge them yourself.

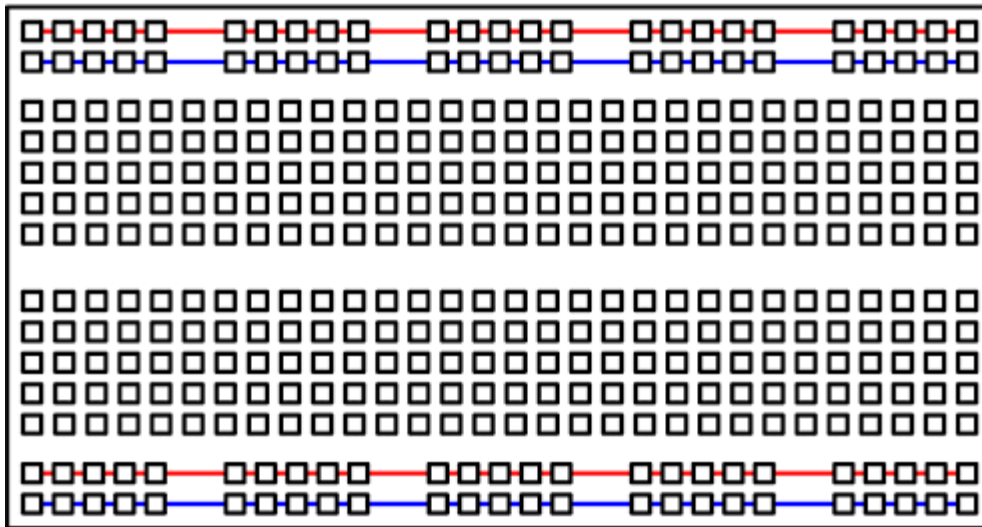
#### REAL-LIFE APPLICATION

Think of a breadboard like a muffin tray. Each 5-hole strip is one muffin cup — anything dropped into the same cup mixes together (electrically connected), but separate cups stay completely separate unless you pour a connection between them with a jumper wire.

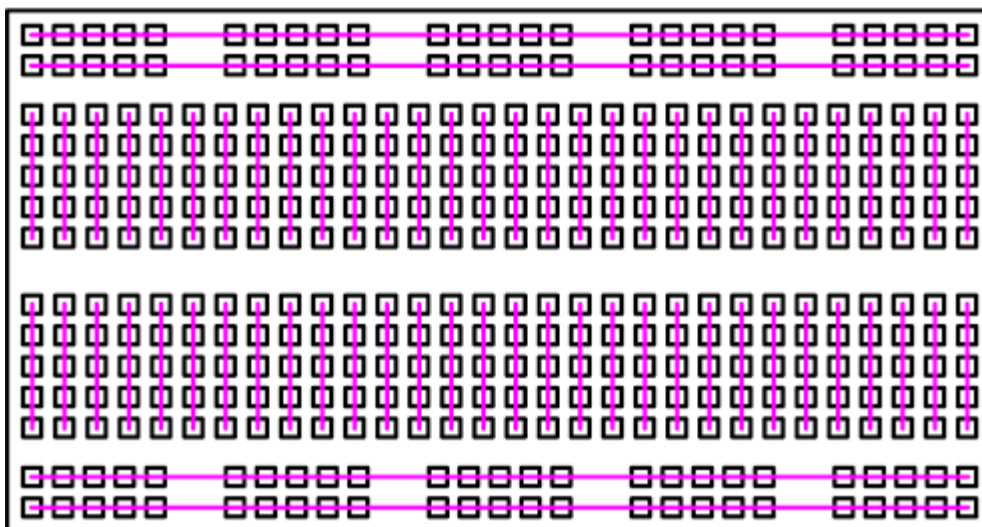




Top View of Breadboard



Interconnect View of Breadboard



## 2.2 Anatomy of an ESP32 Sketch

A program written for the ESP32 is called a sketch, and every sketch is built around exactly two functions:

- `setup()` — runs exactly once, when the board powers on or resets. Used to initialise things: set pin modes, start serial communication, connect to Wi-Fi.
- `loop()` — runs over and over, forever, for as long as the board has power. This is where the ongoing work happens.



#### REAL-LIFE APPLICATION

setup() is like the checklist a pilot runs once before take-off. loop() is like the pilot's in-flight routine — scanning the instruments again and again, for the entire duration of the flight.

## 2.3 Key Embedded Functions

- pinMode(pin, OUTPUT) — tells the ESP32 that a given pin will send voltage out, rather than read it in.
- digitalWrite(pin, HIGH/LOW) — sends 3.3V (HIGH) or 0V (LOW) to a pin.
- delay(ms) — pauses the program for the given number of milliseconds before continuing.

#### REAL-LIFE APPLICATION

digitalWrite(HIGH/LOW) works exactly like a wall light switch — flipping it one way sends voltage down the wire and lights the bulb; flipping it the other way cuts the voltage and the bulb goes dark.

## 2.4 The Blink Program

Bringing everything together — variables, functions, and the setup()/loop() structure — here is your first full Embedded C program:

```
// Session 1 & 2 Project: Blink the ESP32 onboard LED
// Mamuza Engineering - IoT Awareness Program

#define LED_PIN 2    // Change this if your board's LED is on a different pin

void setup() {
    pinMode(LED_PIN, OUTPUT);    // Tell the ESP32 this pin will send OUTPUT signals
    Serial.begin(115200);        // Start serial communication for debugging
    Serial.println("LED Blink program started!");
}

void loop() {
    digitalWrite(LED_PIN, HIGH); // Turn the LED ON
    Serial.println("LED ON");
    delay(1000);                 // Wait 1 second

    digitalWrite(LED_PIN, LOW);  // Turn the LED OFF
    Serial.println("LED OFF");
    delay(1000);                 // Wait 1 second
}
```

### Line-by-Line: What's Actually Happening

| Line                        | What It Does                                                                                                             |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------|
| #define LED_PIN 2           | Creates a name (LED_PIN) for pin number 2, making the code easier to read and change later.                              |
| pinMode(LED_PIN, OUTPUT)    | Configures GPIO 2 as an output pin, so it can send voltage out.                                                          |
| Serial.begin(115200)        | Opens a communication channel to your computer at 115200 bits per second, so you can see messages in the Serial Monitor. |
| digitalWrite(LED_PIN, HIGH) | Sends 3.3V to the pin, lighting the LED.                                                                                 |

|                                         |                                                                        |
|-----------------------------------------|------------------------------------------------------------------------|
| <code>delay(1000)</code>                | Pauses the program for 1000 milliseconds (1 second) before continuing. |
| <code>digitalWrite(LED_PIN, LOW)</code> | Sends 0V to the pin, turning the LED off.                              |

#### REAL-LIFE APPLICATION

This exact pattern — turn something ON, wait, turn it OFF, wait, repeat — is the literal logic behind traffic light controllers and the blinking hazard indicators in a car.

## 2.5 Upload Steps

- Plug in your ESP32 via USB and select the correct Board and Port (Tools menu).
- Copy the code above into a new sketch in the Arduino IDE.
- Click the Upload (→) button and watch the console for "Done uploading."
- Watch your ESP32 — the onboard LED should now blink once per second.
- Open Tools → Serial Monitor (baud rate 115200) to see the "LED ON" / "LED OFF" messages print live.

#### CHECKPOINT

If your LED is blinking and you can see messages in the Serial Monitor — congratulations. You've written and run your first Embedded C program, and completed the Perception Layer's most basic building block: controlling an output.

## Today's Recap

---

- C is a fast, low-level language that gives us close control over hardware — exactly why it's used to program the ESP32.
- Comments explain your code to humans and are ignored by the compiler.
- Variables are labelled containers, and their data type (int, float, char, bool) decides what they can hold.
- Statements are single instructions; operators (arithmetic, comparison, logical, assignment) act on your data.
- Functions are reusable blocks of code — and `setup()` / `loop()` are two special, required functions in every ESP32 sketch.
- `if` / `else` statements let your program make decisions; `for` and `while` loops let it repeat actions.
- A breadboard's outer rails carry power the length of the board, while its inner strips connect components in groups of 5 holes.
- You wrote, uploaded, and ran your first Embedded C program: the Blink sketch.

### Looking Ahead

In our next session, we move from a single fixed output to real inputs — you'll wire push buttons and a touch sensor, read a potentiometer, and use PWM to mix RGB colours.

